STAJ PROJESİ · TÜRKSAT AVIATION AND COMMUNICATION

Akıllı Okul Bilgi Sistemi

Hafta 4 — Doğal Dil → SQL

HAZIRLAYAN	Bera Ergin — s29315
KURUM	PJAIT — 7. Yarıyıl
BU HAFTA	İlk gerçek veri döndüren uç nokta: POST /query

Nereden Geliyoruz, Nereye Gidiyoruz

HAFTA 3 — BİTTİ

- ✓ Bağlam parametreli kuruluyor
- ✓ İstekler arası sızıntı yok
- ✓ RLS, API'den 4 / 12 / 1 / 1 veriyor
- ! Ama veri döndüren gerçek bir uç nokta yok

Yalnızca geçici bir /debug/students uç noktası var.

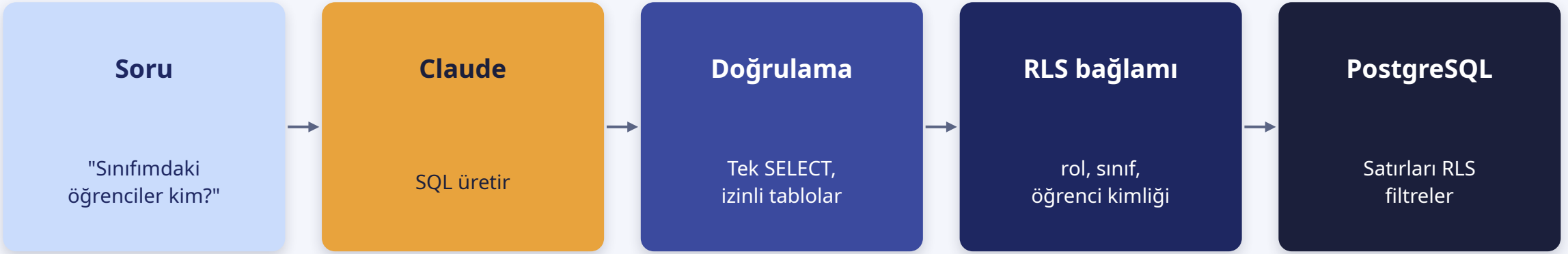
HAFTA 4 — HEDEF

- POST /query ile doğal dilde soru
- Claude SQL üretir
- SQL doğrulanır, olduğu gibi çalışır
- Filtrelemeyi RLS yapar, SQL değiştirilmez

Niyet analizi, açıklama ve rate limit sonraki haftaya kalıyor.

Sorgu Akışı

Bir sorunun cevaba dönüşene kadar geçtiği aşamalar



Kritik tasarım kararı

Claude'un ürettiği SQL değiştirilmiyor, sarmalanmıyor, ayrıştırılıp yeniden yazılmıyor. Doğrulama yalnızca "bu SQL çalıştırılabilir mi" sorusuna bakıyor. "Hangi satırları görebilir" sorusunu, sorgunun şekli ne olursa olsun, veritabanının RLS politikaları cevaplıyor. Takma ad, JOIN, toplama, alt sorgu: hepsi aynı şekilde filtreleniyor.

Üç Savunma Katmanı

Her girişimi, farklı bir katman durduruyor

`DROP TABLE students`	Doğrulama	SELECT olmayan ifade, 400
`SELECT * FROM users`	Doğrulama + yetki	İzinsiz tablo, 400. Doğrulama atlansa bile veritabanı yetkiyi reddeder
`SELECT * FROM students (öğretmen)`	RLS	Yalnızca kendi sınıfının 4 satırı döner
`SELECT * FROM students WHERE 1=1`	RLS	Koşul hile yapmaz, RLS yine filtreler

school_app rolü yalnızca students, grades, attendance üzerinde SELECT yetkisine sahip; users ve teachers tablolarında hiçbir yetkisi yok (en az yetki ilkesi).

Kaynak Sınırları ve Hata Yönetimi

Claude'un ürettiği SQL ne kadar kötü olursa olsun sunucu ayakta kalmalı

Süre sınırı	Satır sınırı	SQL gizli	Yoğunlukta 503
Her sorgu en fazla 5 saniye çalışır (statement_timeout)	En fazla 500 satır okunur	Üretilen SQL yanıta konmaz, yalnızca sunucu loguna yazılır	API yoğunsa çıplak 500 yerine anlaşılır bir 503 döner

Bir model, çok pahalı bir sorgu (örneğin kartezyen çarpım) yazabilir. RLS satırları filtreler ama sorgunun maliyetini sınırlamaz; bu yüzden süre sınırı ayrı bir önlem olarak konuyor.

Aynı SQL, Dört Rol

Sorgu hiç değişmiyor; yalnızca oturumun bağlamı farklı

Rol	`SELECT * FROM students`	`SELECT COUNT(*) AS total FROM students`
`principal`	**12 satır**	**total = 12**
`teacher (9-A)`	**4 satır**	**total = 4**
`parent`	**1 satır**	**total = 1**
`student`	**1 satır**	**total = 1**

Toplama sorguları da doğru sayıyı veriyor çünkü filtre, sonuca değil tabloya uygulanıyor; bunu veritabanı kendisi yapıyor.

API Anahtarı Olmadan Test

Claude'a bağımlı kod, Claude olmadan doğrulanabiliyor

İki teknik

`translate_question_to_sql(question, client=None)`

Sahte bir client verilebiliyor; gerçek çağrı yapılmıyor.

monkeypatch ile /query testlerinde üretilen SQL sabitleniyor.

Kanıtlananlar

- Aynı SQL: 4 / 12 / 1 / 1 satır
- users erişimi: 400 ve veritabanı yetki hatası
- DROP ve çoklu ifade reddi
- 529 hatası, 503'e çevriliyor

Yeni bağımlılıklar: anthropic (Claude API) ve sqlglot (SQL doğrulaması için ayrıştırıcı). Anahtar yalnızca .env'den okunur, kodda ve repoda yer almaz.

Hafta 4 — Kapsam ve Kabul Kriterleri

Bu hafta teslim edilenler

- ✓ llm_service.py — doğal dilden SQL
- ✓ sql_safety.py — sqlglot ile doğrulama
- ✓ query.py — POST /query
- ✓ school_app için en az yetki
- ✓ /debug/students kaldırıldı

Kabul kriterleri

1. Geçerli token ile POST /query 200 ve sonuç döner
2. Aynı SQL dört rolde 12 / 4 / 1 / 1 satır verir
3. users/teachers erişimi 400; veritabanı da reddeder
4. 5 saniyeyi aşan sorgu iptal olur, en fazla 500 satır
5. Üretilen SQL yanıtta yok; 529 → 503

Sonraki Adım — Hafta 5

Niyet Analizi, Açıklanabilirlik ve Sertleştirme

- Sohbet mi veri sorgusu mu: niyet tespiti
- Sonuçla birlikte "neden bu sonuç" açıklaması
- Rate limiting ve yapılandırılmış loglama
- Prompt iyileştirmeleri: isim araması, kimlik sütunları

Teşekkürler